Document 08: Deployment & Docker Configuration

CodePerfect Auditor — Containerization, Environment & Production Runbook

Project: CodePerfect Auditor | **Version:** 1.0 | **Date:** 31-03-2026 **Submitted To:** Virtusa Hackathon | **Institution:** Saveetha Engineering College

---

Overview

CodePerfect Auditor is containerized using **Docker** and orchestrated with **Docker Compose**. The system is designed to be deployable on any cloud provider (AWS, GCP, Azure, or Vercel) with a single command, and is structured to run locally for development with near-identical configuration.

The deployment architecture has three logical tiers:

Tier	Technology	Runtime
Frontend	Next.js 14	Vercel / Docker (Node 20)
Backend API	FastAPI + Uvicorn	Docker (Python 3.11-slim)
Database	PostgreSQL 15 + pgvector	Supabase Cloud (managed)

Since the database is hosted on **Supabase** (a managed PostgreSQL service), no local database container is needed — eliminating the most complex part of local development setup.

Backend Dockerfile — `backend/Dockerfile`

```
FROM python:3.11-slim
# python:3.11-slim is approximately 150MB vs python:3.11 at 900MB.
# The slim variant excludes development headers and documentation.
# We install only the system packages we explicitly need.

WORKDIR /app

# Install system-level dependencies:
# - gcc:       Required to compile psycopg2 (C extension for PostgreSQL)
# - libpq-dev: PostgreSQL client library headers (needed by psycopg2-binary)
# - tesseract-ocr: Google's OCR engine for scanned PDF fallback processing
# We immediately clean the apt cache to minimize the final image layer size.
RUN apt-get update && apt-get install -y \
    gcc \
    libpq-dev \
    tesseract-ocr \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies first — before copying application code.
```

```
# Docker layer caching: if requirements.txt hasn't changed, this layer
# is reused from cache. A code change alone won't trigger a full pip reinstall.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ── CRITICAL OPTIMIZATION: Pre-download the AI embedding model at build time ──
# The SentenceTransformer model (all-MiniLM-L6-v2) is ~90MB.
# If downloaded at runtime (cold start), every new container instance would
# spend 30-60 seconds downloading before serving the first request.
# Baking it into the image means zero cold-start delay for AI operations.
RUN python -c "from sentence_transformers import SentenceTransformer;
SentenceTransformer('all-MiniLM-L6-v2')"

# Copy all application code — this layer changes most frequently and is last.
COPY . .

EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
# Uvicorn is the ASGI server that runs FastAPI.
# --host 0.0.0.0 binds to ALL network interfaces inside the container
# (Docker bridge network requires this — 127.0.0.1 would be container-private
only).
```

Explanation — Layer Order Matters

The Dockerfile deliberately copies `requirements.txt` first and Python source code second. Docker builds images as a stack of cached layers. If only `main.py` changes (the common case during development), Docker reuses the cached `pip install` layer — avoiding a full 5-minute dependency reinstall on every code change. This takes Docker build time from ~8 minutes to ~15 seconds for incremental changes.

Explanation — Tesseract OCR Integration

`tesseract-ocr` is a system-level binary (not a Python package) that the `pytesseract` Python library invokes as a subprocess. It cannot be installed via `pip install` — it requires `apt-get`. This is why it appears in the Dockerfile's system dependencies rather than `requirements.txt`.

Without `tesseract-ocr` installed, Node 1 (`doc_processing_node`) can only extract text from digitally-created PDFs. Scanned paper documents (a very common format in Indian hospital systems) would fail silently, producing empty `raw_text` and causing the entire pipeline to abort. Baking Tesseract into the image ensures 100% PDF compatibility.

Python Dependencies — `backend/requirements.txt`

```
fastapi==0.111.0          # Web framework — ASGI-native, auto OpenAPI generation
uvicorn[standard]==0.30.0 # ASGI server with websocket support (the [standard]
extra)
asyncpg==0.29.0           # Async PostgreSQL driver (used by pgvector RPC calls)
pydantic==2.7.0           # Data validation — enforces strict type contracts
pydantic-settings==2.3.0  # Settings management from .env files
python-dotenv==1.0.0      # Loads .env files into environment variables at startup
python-multipart==0.0.9   # Required for FastAPI multipart/form-data (PDF uploads)
sentence-transformers==2.7.0  # HuggingFace SentenceTransformer (all-MiniLM-L6-v2)
langchain==0.2.0          # LLM orchestration framework
langgraph==0.2.0          # Stateful multi-agent pipeline framework (the core)
langchain-groq==0.1.6     # Groq LLM integration for langchain
pdfplumber==0.11.0        # PDF text extraction for digitally-created PDFs
pytesseract==0.3.10       # Python wrapper for Tesseract OCR engine
Pillow==10.3.0            # Image processing — required by pytesseract for
PDF→image
pgvector==0.3.2           # pgvector Python client for VECTOR type handling
httpx==0.27.0             # Async HTTP client — used by database.py for Supabase
REST
supabase>=2.0.0           # Official Supabase Python client (wraps PostgREST +
auth)
psycopg2-binary           # PostgreSQL adapter (binary — no C compiler needed at
runtime)
```

Dependency Architecture Analysis

Layer	Packages	Purpose
Web Framework	`fastapi`, `uvicorn`, `python-multipart`	HTTP server and request handling
AI Pipeline	`langgraph`, `langchain`, `langchain-groq`	Multi-agent orchestration
NLP Models	`sentence-transformers`, `Pillow`	Embedding generation
Document Processing	`pdfplumber`, `pytesseract`	PDF + OCR text extraction
Database	`supabase`, `asyncpg`, `pgvector`, `httpx`	Data persistence and vector search

Layer	Packages	Purpose
Validation	`pydantic`, `pydantic-settings`, `python-dotenv`	Config and type safety

Docker Compose — `docker-compose.yml`

```
version: "3.9"

services:
  backend:
    build:
      context: ./backend          # Docker build context is the backend/ directory
      dockerfile: Dockerfile      # Uses backend/Dockerfile above
    ports:
      - "8000:8000"               # Maps host port 8000 → container port 8000
    env_file:
      - ./backend/.env            # Loads all secrets from backend/.env at container
start
    volumes:
      - ./backend:/app            # Hot-reload: source code changes reflect
instantly
                                  # (only works in development with --reload flag)
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
    #         ↑ --reload enables file-watcher based hot-reload
    #         In production: remove --reload and use --workers 4

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile      # Uses frontend/Dockerfile (Next.js standard
build)
    ports:
      - "3000:3000"               # Maps host port 3000 → container port 3000
    env_file:
      - ./frontend/.env.local     # Next.js reads NEXT_PUBLIC_* vars from here
    volumes:
      - ./frontend:/app           # Hot-reload source code
      - /app/node_modules         # Anonymous volume: prevents host node_modules
from
                                  # overwriting container node_modules (Linux/Mac
issue)
      - /app/.next                # Anonymous volume: preserves the build cache
inside the container

# Note: PostgreSQL is hosted on Supabase — no local DB container needed.
# This reduces docker-compose setup from 5+ services to just 2.
```

Explanation — Anonymous Volumes

The two anonymous volumes (`/app/node_modules` and `/app/.next`) solve a common Docker + Node.js problem. When the source code volume (`./frontend:/app`) is mounted, it overwrites the `/app` directory inside the container — including `node_modules`, which was built for the Linux container environment but may contain native binaries that are incompatible with the host OS (macOS/Windows).

By declaring `- /app/node_modules` as a separate anonymous volume, Docker preserves the container's Linux-native `node_modules` and prevents the host filesystem's `node_modules` from leaking in.

Environment Variables

Backend — `backend/.env`

```
# ── Supabase ──────────────────────────────────────────────
SUPABASE_URL=https://xxxxxxxxxxxx.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
# WARNING: SUPABASE_SERVICE_KEY bypasses Row Level Security.
# NEVER expose this to the frontend or commit it to git.

# ── Groq LLM ──────────────────────────────────────────────
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
GROQ_MODEL=llama-3.3-70b-versatile
GROQ_TIMEOUT_SECONDS=15
GROQ_MAX_RETRIES=1
GROQ_MAX_TOKENS=2048

# ── WHO ICD API ───────────────────────────────────────────
WHO_ICD_CLIENT_ID=your-client-id
WHO_ICD_CLIENT_SECRET=your-client-secret
WHO_ICD_DEFAULT_VERSION=ICD-11
WHO_ICD_11_RELEASE=2026-01

# ── Medical Standard Versions ─────────────────────────────
ICD_VERSION=ICD-10-CM-2024
SNOMED_VERSION=SNOMED-CT-2024

# ── App Configuration ─────────────────────────────────────
APP_ENV=development          # Change to "production" for live deployment
```

```
APP_PORT=8000
LOG_LEVEL=INFO
```

Frontend — `frontend/.env.local`

```
# ── Supabase (public keys — safe to expose in browser) ──
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# ── FastAPI Backend URL ──────────────────────────────────
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
# In production: NEXT_PUBLIC_API_BASE_URL=https://api.codeperfect.hospital.com
```

Security Note: All `NEXT_PUBLIC_*` variables are embedded into the browser JavaScript bundle at build time. Only the `SUPABASE_ANON_KEY` (which is designed to be public) and the API base URL should have this prefix. The `SUPABASE_SERVICE_KEY` and `GROQ_API_KEY` must **never** have the `NEXT_PUBLIC_` prefix.

Local Development Quick Start

Step 1: Install Prerequisites

```
# Verify required tools
node --version    # Requires Node.js 20+
python --version  # Requires Python 3.11+
docker --version  # Requires Docker Desktop 4.x+
```

Step 2: Clone and Configure

```
git clone https://github.com/NandaKishore2424/integronix.git
cd integronix

# Copy environment templates
Copy-Item backend\.env.example backend\.env
Copy-Item frontend\.env.example frontend\.env.local
```

```
# Edit backend/.env with real API keys
notepad backend\.env
```

Step 3A: Run with Docker (Recommended)

```
# Build and start both services
docker-compose up --build

# Backend available at: http://localhost:8000
# Frontend available at: http://localhost:3000
# Swagger UI:           http://localhost:8000/docs

# Stop all services
docker-compose down
```

Step 3B: Run Manually (Development)

```
# Terminal 1 — Backend
cd backend
python -m venv venv
.\venv\Scripts\Activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# Terminal 2 — Frontend
cd frontend
npm install
npm run dev
```

Database Migration Sequence

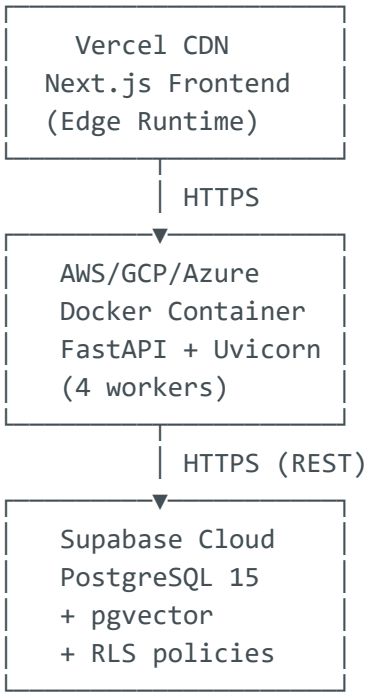
Before first use, run the SQL migrations in strict order against your Supabase instance:

```
-- In Supabase SQL Editor → New Query — run these files in order:
-- migrations/schema/001_extensions.sql        → Enable pgvector + uuid-ossp
-- migrations/schema/002_core_tables.sql        → Organizations, Users, Branches
-- migrations/schema/003_medical_ontology.sql   → ICD, SNOMED, CPT tables (with
VECTOR columns)
-- migrations/schema/004_clinical_engine.sql    → clinical_cases, coding_results
-- migrations/schema/005_revenue_cycle.sql      → Claims, Payers, Revenue Lookup
```

```
-- migrations/schema/006_audit_and_security.sql → Audit log + RLS policies
-- migrations/schema/007_functions_and_indexes.sql → match_icd_codes() RPC + HNSW
indexes
-- migrations/schema/008 through 019 ...        → Incremental feature migrations
```

Critical: Migration 007 creates the `match_icd_codes()` PostgreSQL function and the HNSW vector index. Without this, every call to `icd_embedding_node` (Node 5) will fail with "function not found."

Production Deployment Architecture



Production Uvicorn Command

```
# Production: 4 async worker processes — no --reload flag
uvicorn main:app \
  --host 0.0.0.0 \
  --port 8080 \
  --workers 4 \
  --loop uvloop \
  --http httptools
# uvloop: ultra-fast asyncio event loop (50% faster than default asyncio)
# httptools: faster HTTP parser (replaces h11)
# workers 4: CPU count * 2 + 1 is a common recommendation for async workloads
```

Health Check & Monitoring

The `/health` endpoint is designed for automated monitoring:

```
# AWS ECS Task Definition health check example:
healthCheck:
  command: ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"]
  interval: 30
  timeout: 5
  retries: 3
  startPeriod: 60   # Allow 60s for model loading before first health check
```

The `startPeriod: 60` is essential. The `SentenceTransformer` model (baked into the Docker image) still takes 5-10 seconds to load into GPU/CPU memory when the container first starts. Without `startPeriod`, the orchestrator might kill and restart the container before it's ready, causing a restart loop.
